

Constrained Update for Continual Reinforcement Learning

Setup

Tasks $1, \dots, k$ have appeared; k is the current task and $i \in \{1, \dots, k\}$ a generic index. Task i is the MDP

$$M_i = (S, A, P_i, r_i, \rho_i, \gamma), \quad (1)$$

with states S , actions A , transition law $P_i(s' | s, a)$, reward $r_i(s, a)$, initial distribution ρ_i , and discount $\gamma \in (0, 1)$.

The local policy is $\pi_L = \pi_\theta$ and the global policy is $\pi_G = \pi_\phi$, with parameters θ and ϕ . The two are trained in alternation: during a local update, θ is optimized while π_G is held fixed; during a global update, ϕ is optimized while π_L is held fixed. Each local update starts from the global,

$$\theta^{(0)} = \phi. \quad (2)$$

We use uniform task weights for now, $\omega_i = 1/k$.

A length- T trajectory is $\tau = (s_0, a_0, \dots, s_T, a_T, s_{T+1})$, with law under policy π on task i

$$p_i^\pi(\tau) = \rho_i(s_0) \prod_{t=0}^T \pi(a_t | s_t) P_i(s_{t+1} | s_t, a_t). \quad (3)$$

The discounted return and the value of π on task i are

$$G_i(\tau) = \sum_{t=0}^T \gamma^t r_i(s_t, a_t), \quad V_i^\pi = \mathbb{E}_{\tau \sim p_i^\pi} [G_i(\tau)]. \quad (4)$$

We assume access to all previously seen environments. Thus old-task terms are estimated from fresh rollouts in old environments rather than from a stored replay buffer; the method is replay-free in that sense.

The objective and the constraint

We score a policy by the weighted sum of its per-task values, the weights $\omega_i \geq 0$ fixing how much each task counts,

$$J_k(\pi) = \sum_{i=1}^k \omega_i V_i^\pi. \quad (5)$$

For a new policy π_{new} and a reference policy π_{ref} , the total improvement of one policy over another can be split into current-task improvement and past-task improvement:

$$\begin{aligned} J_k(\pi_{\text{new}}) - J_k(\pi_{\text{ref}}) &= \sum_{i=1}^k \omega_i V_i^{\pi_{\text{new}}} - \sum_{i=1}^k \omega_i V_i^{\pi_{\text{ref}}} \\ &= \sum_{i=1}^k \omega_i (V_i^{\pi_{\text{new}}} - V_i^{\pi_{\text{ref}}}) \\ &= \underbrace{\omega_k (V_k^{\pi_{\text{new}}} - V_k^{\pi_{\text{ref}}})}_{\text{current task}} + \underbrace{\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_{\text{new}}} - V_i^{\pi_{\text{ref}}})}_{\text{past tasks}}. \end{aligned} \quad (6)$$

The local model is intentionally left unconstrained so that it can fully adapt to the current task. Past-task preservation is enforced only during the subsequent global update. During the global update, the local policy serves as the reference for the current task, while the global policy is encouraged to retain performance on previously learned tasks.

Local objective

The local model updates π_L with the global held fixed. Each local update is initialized from the current global policy according to (2). Empirically, constraining the local model reduced its ability to sufficiently learn the current task. Therefore, the local model is left unconstrained and is optimized solely to maximize its expected return on the current task. Any forgetting introduced during this stage is addressed during the subsequent global update.

The local objective is

$$J_L(\pi_L) = V_k^{\pi_L}, \quad (7)$$

and the corresponding optimization problem is

$$\max_{\pi_L} J_L(\pi_L), \quad (8)$$

where the optimization is performed only on the current task.

Global objective

The global model updates π_G with the local held fixed. It maximizes its past lead over the local and is kept within tolerance $\epsilon \geq 0$ of the local on the current task. The same one-sided idea is used here: the global model is penalized only when its current-task value is below the local model's current-task value. With the current-task shortfall

$$F_G(\pi_G) = \begin{cases} (V_k^{\pi_L} - V_k^{\pi_G})^2, & \text{if } V_k^{\pi_G} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_G} \geq V_k^{\pi_L}, \end{cases} \quad (9)$$

the global problem, its Lagrangian, and its saddle point are

$$\max_{\pi_G} \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G} - V_i^{\pi_L}) \quad \text{s.t.} \quad F_G(\pi_G) \leq \epsilon, \quad (10)$$

$$\mathcal{L}_G(\pi_G, \mu) = \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G} - V_i^{\pi_L}) - \mu (F_G(\pi_G) - \epsilon), \quad (11)$$

$$\max_{\pi_G} \min_{\mu \geq 0} \mathcal{L}_G(\pi_G, \mu). \quad (12)$$

During alternation, the local update maximizes performance on the current task without constraints. The subsequent global update consolidates this newly acquired knowledge while preserving performance on previous tasks through constrained optimization.

Notes for experiments. A scalar value constraint may not always imply that the global policy has absorbed the local policy's behavior; this should be checked empirically before adding a distillation term. We should also monitor feasibility: persistent constraint violations or diverging multipliers indicate task conflict or insufficient model capacity. Since the old-task reference is the current global value, cumulative forgetting should be checked in final per-task evaluation.

Local update

Optimizing over the local policy means optimizing over its parameters θ , since $\pi_L = \pi_\theta$. The local objective is unconstrained and is optimized by gradient ascent on $J_L(\theta)$. The update requires the policy gradient $\nabla_\theta V_k^{\pi_\theta}$, which is derived below. Since the derivation applies to an arbitrary task, we first derive $\nabla_\theta V_i^{\pi_\theta}$ for a generic task i .

The chain rule on the logarithm gives $\nabla_\theta \log p_i^{\pi_\theta}(\tau) = \nabla_\theta p_i^{\pi_\theta}(\tau) / p_i^{\pi_\theta}(\tau)$, and multiplying both sides by $p_i^{\pi_\theta}(\tau)$,

$$\nabla_\theta p_i^{\pi_\theta}(\tau) = p_i^{\pi_\theta}(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau).$$

With finite S and A the value (4) is a sum over all trajectories, $V_i^{\pi_\theta} = \sum_\tau p_i^{\pi_\theta}(\tau) G_i(\tau)$. Substituting the identity,

$$\begin{aligned} \nabla_\theta V_i^{\pi_\theta} &= \nabla_\theta \sum_\tau p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \sum_\tau \nabla_\theta p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \sum_\tau p_i^{\pi_\theta}(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \mathbb{E}_{\tau \sim p_i^{\pi_\theta}} [G_i(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau)]. \end{aligned} \quad (13)$$

In the trajectory law (3) only the policy factors depend on θ , so ρ_i and P_i drop from the log-gradient,

$$\begin{aligned}
\nabla_\theta \log p_i^{\pi_\theta}(\tau) &= \nabla_\theta \left[\log \rho_i(s_0) + \sum_{t=0}^T \log \pi_\theta(a_t | s_t) + \sum_{t=0}^T \log P_i(s_{t+1} | s_t, a_t) \right] \\
&= \underbrace{\nabla_\theta \log \rho_i(s_0)}_0 + \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) + \underbrace{\sum_{t=0}^T \nabla_\theta \log P_i(s_{t+1} | s_t, a_t)}_0 \\
&= \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t).
\end{aligned} \tag{14}$$

Substituting (14) into (13) and factoring out the return, which does not carry the index t , gives the per-task policy gradient

$$\nabla_\theta V_i^{\pi_\theta} = \mathbb{E}_{\tau \sim p_i^{\pi_\theta}} \left[G_i(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \tag{15}$$

During the local update, the optimization objective is

$$J_L(\theta) = V_k^{\pi_\theta}, \tag{16}$$

Differentiating the objective with respect to θ gives

$$\nabla_\theta J_L(\theta) = \nabla_\theta V_k^{\pi_\theta}. \tag{17}$$

Substituting the policy gradient from (15),

$$\nabla_\theta J_L(\theta) = \mathbb{E}_{\tau \sim p_k^{\pi_\theta}} \left[G_k(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \tag{18}$$

Using the Monte Carlo estimator,

$$\widehat{V}_k^{\pi_\theta} = \frac{1}{N} \sum_{n=1}^N G_k(\tau^{(n)}), \tag{19}$$

and

$$\overline{\nabla_\theta V_k^{\pi_\theta}} = \frac{1}{N} \sum_{n=1}^N G_k(\tau^{(n)}) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(n)} | s_t^{(n)}), \tag{20}$$

the local parameters are updated by gradient ascent,

$$\begin{aligned}
\theta^{(m+1)} &= \theta^{(m)} + \alpha \nabla_\theta J_L(\theta) \\
&= \theta^{(m)} + \alpha \overline{\nabla_\theta V_k^{\pi_\theta}}.
\end{aligned} \tag{21}$$

Global update

Optimizing over the global policy means optimizing over its parameters ϕ , since $\pi_G = \pi_\phi$. A saddle point of (12) is approached by ascending $\mathcal{L}_G$ in ϕ and descending it in μ . Both directions need $\nabla_\phi \mathcal{L}_G$, built from the per-task gradients $\nabla_\phi V_i^{\pi_\phi}$. The derivation (13)–(15) used nothing about θ beyond the fact that only the policy factors of the trajectory law (3) carry the parameter, so it holds verbatim with ϕ in place of θ ,

$$\nabla_\phi V_i^{\pi_\phi} = \mathbb{E}_{\tau \sim p_i^{\pi_\phi}} \left[G_i(\tau) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t) \right]. \tag{22}$$

During the global update, the local policy is held fixed. Therefore,

$$\nabla_\phi V_i^{\pi_L} = 0, \quad \nabla_\phi V_k^{\pi_L} = 0, \quad \nabla_\phi \epsilon = 0.$$

For the squared shortfall in (9),

$$\begin{aligned}
\nabla_\phi F_G(\pi_\phi) &= \nabla_\phi (V_k^{\pi_L} - V_k^{\pi_\phi})^2 \\
&= 2 (V_k^{\pi_L} - V_k^{\pi_\phi}) \nabla_\phi (V_k^{\pi_L} - V_k^{\pi_\phi}) \\
&= 2 (V_k^{\pi_L} - V_k^{\pi_\phi}) (0 - \nabla_\phi V_k^{\pi_\phi}) \\
&= -2 (V_k^{\pi_L} - V_k^{\pi_\phi}) \nabla_\phi V_k^{\pi_\phi},
\end{aligned} \tag{23}$$

$$\nabla_{\phi} F_G(\pi_{\phi}) = \begin{cases} -2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}, & \text{if } V_k^{\pi_{\phi}} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_{\phi}} \geq V_k^{\pi_L}. \end{cases} \quad (24)$$

Differentiating the Lagrangian (11) with respect to ϕ ,

$$\begin{aligned} \nabla_{\phi} \mathcal{L}_G &= \nabla_{\phi} \left[\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_{\phi}} - V_i^{\pi_L}) - \mu (F_G - \epsilon) \right] \\ &= \sum_{i=1}^{k-1} \omega_i (\nabla_{\phi} V_i^{\pi_{\phi}} - \nabla_{\phi} V_i^{\pi_L}) - \mu (\nabla_{\phi} F_G - \nabla_{\phi} \epsilon) \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} - \mu \nabla_{\phi} F_G \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} - \mu [-2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}] \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} + 2\mu (V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}. \end{aligned} \quad (25)$$

since both the local policy and the tolerance are fixed during the global update.

$$\nabla_{\phi} \mathcal{L}_G(\pi_{\phi}, \mu) = \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} + \mu \begin{cases} 2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}, & \text{if } V_k^{\pi_{\phi}} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_{\phi}} \geq V_k^{\pi_L}, \end{cases} \quad (26)$$

each per-task gradient given by (22).

Drawing N trajectories $\tau^{(1)}, \dots, \tau^{(N)}$ on task i under π_{ϕ} , the per-task value and gradient are estimated by

$$\hat{V}_i^{\pi_{\phi}} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}), \quad (27)$$

and

$$\overline{\nabla_{\phi} V_i^{\pi_{\phi}}} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}) \sum_{t=0}^T \nabla_{\phi} \log \pi_{\phi}(a_t^{(n)} | s_t^{(n)}). \quad (28)$$

During the same global update, $\hat{V}_k^{\pi_L}$ denotes the rollout estimate of the held-fixed local value on the current task. The global primal update is

$$\phi^{(m+1)} = \phi^{(m)} + \beta \left[\sum_{i=1}^{k-1} \omega_i \overline{\nabla_{\phi} V_i^{\pi_{\phi}}} + \mu^{(m)} \begin{cases} 2(\hat{V}_k^{\pi_L} - \hat{V}_k^{\pi_{\phi}}) \overline{\nabla_{\phi} V_k^{\pi_{\phi}}}, & \text{if } \hat{V}_k^{\pi_{\phi}} < \hat{V}_k^{\pi_L}, \\ 0, & \text{if } \hat{V}_k^{\pi_{\phi}} \geq \hat{V}_k^{\pi_L}, \end{cases} \right]. \quad (29)$$

The multiplier is obtained by minimizing the Lagrangian with respect to μ . Since

$$\frac{\partial \mathcal{L}_G}{\partial \mu} = -(F_G - \epsilon), \quad (30)$$

gradient descent on μ gives

$$\begin{aligned} \mu^{(m+1)} &= \mu^{(m)} - \eta_{\mu} \frac{\partial \mathcal{L}_G}{\partial \mu} \\ &= \mu^{(m)} + \eta_{\mu} (F_G - \epsilon). \end{aligned} \quad (31)$$

Finally, because the Lagrange multiplier must satisfy $\mu \geq 0$, it is projected onto the nonnegative orthant,

$$\mu^{(m+1)} = \max \{0, \mu^{(m)} + \eta_{\mu} (\hat{F}_G^{(m)} - \epsilon)\}, \quad (32)$$

where

$$\hat{F}_G^{(m)} = \begin{cases} (\hat{V}_k^{\pi_L} - \hat{V}_k^{\pi_{\phi}^{(m)}})^2, & \text{if } \hat{V}_k^{\pi_{\phi}^{(m)}} < \hat{V}_k^{\pi_L}, \\ 0, & \text{if } \hat{V}_k^{\pi_{\phi}^{(m)}} \geq \hat{V}_k^{\pi_L}. \end{cases} \quad (33)$$

After the global phase for task k , the new task becomes part of the past-task set for the next task.

When we have the expert models stored

We can save a copy of the expert models to generate the values $V_i^\star(s)$ for the past tasks. The idea behind doing so is to evaluate the values of past tasks without an intensive rollout calculation. However, we do not require saving both the actor and the critic. Saving only the critic serves our purpose. During the consolidation phase only the actor is updated, and the critic remains fixed. We store the critic so that we can generate $V_i^\star(s)$ on the i -th task. Later on we use these values to compare against the global model’s performance $V_i^{\pi_g}(s)$, and thus train the global model. However, to achieve this we cannot share a trunk between the actor and the critic. In such a case the critic is also updated during the consolidation phase, and the value $V_i^{\pi_g}$ read off that critic is no longer faithful. While implementing this project, Claude introduced this mistake in our code.

If we are saving the expert critic model, then we cannot formulate this problem in a min-max setup. The min-max formulation came from the assumption that the current local model is bad at past tasks but expert on the current task, with the situation flipped for the global model. Once we have access to every past expert, there is no longer a notion of the local model being bad, because the local model is simply the expert of the current task. We only have to improve the global model through a loss built from the difference between $V_i^\star$ and $V_i^{\pi_g}$. In our current implementation we are using the best local experts inside the min-max formulation, which is inconsistent and therefore incorrect.

To faithfully train the global model on both past and current tasks we need to generate enough evaluation states, for instance 50 no-op states together with 50 random intermediate states per task. Whether that number is enough has to be checked. The expert side of the comparison is now cheap because the fixed critics supply $V_i^\star(s)$ in one forward pass, so expert rollouts disappear entirely. The global side of the comparison still needs samples, which is discussed in § *Cost of the estimator* below.

Everything below is stated once so that the expert quantities and the global quantities never collide. The rule is that a *subscript names the task* and a *superscript names the policy*.

Tasks and dynamics

$M_i = (S_i, A_i, P_i, r_i, \rho_i, \gamma)$	MDP of task i , for $i \in \{1, \dots, k\}$, with k the current task and $\gamma \in (0, 1)$.
$P_i(s' s, a)$	Transition kernel of task i . Capital P is always a kernel.
$p_i^\pi(\tau)$	Trajectory density of policy π on task i , defined in (3). Lowercase p is always a trajectory density.
$p_i^\pi(\tau s)$	Trajectory density conditioned on $s_0 = s$, i.e. (3) with ρ_i removed.
$G_i(\tau) = \sum_t \gamma^t r_i(s_t, a_t)$	Discounted return of τ under the reward of task i .

Policies

$\pi_i^\star = \pi_{\psi_i}$	Expert actor of task i , frozen after task i finishes. Parameters ψ_i are never updated again.
$\pi_g = \pi_\phi$	Global (consolidated) actor, the only trainable policy in this section. Parameters ϕ .
$\pi_L = \pi_\theta$	Local actor. It is the expert-in-training. When task k ends we set $\pi_k^\star \leftarrow \pi_L$, so the local model is not a separate object from the experts.

Values

$V_i^\pi(s)$	State value of policy π on task i , $\mathbb{E}[\sum_t \gamma^t r_i(s_t, a_t) s_0 = s]$.
$V_i^\pi = \mathbb{E}_{s \sim \rho_i}[V_i^\pi(s)]$	Scalar task value, exactly as in (4).
$V_i^\star(s) \approx V_i^{\pi_i^\star}(s)$	Stored expert critic of task i , a frozen network. It is the only place where a $\star$ appears, and it is a constant with respect to ϕ .
$V_i^{\pi_g}(s)$	State value of the global policy on task i . This is the quantity that moves during consolidation.
$\hat{\cdot}, \bar{\cdot}$	Monte Carlo estimate of a value and of a gradient, as in (19)–(20).

Scores

$J_i^\star = V_i^{\pi_i^\star}$	Expert score on task i . A single fixed number, the ceiling for task i . No policy argument, because the policy is pinned to $\pi_i^\star$.
$J_k(\pi) = \sum_{i=1}^k \omega_i V_i^\pi$	Score of one policy across tasks $1, \dots, k$, unchanged from (5). It takes a policy argument, and in this section that argument is always π_g .
$J_{1:k}^\star = \sum_{i=1}^k \omega_i J_i^\star$	Expert ceiling aggregated over the same k tasks. Constant.

Bookkeeping

$\omega_i \geq 0$	Task weights, currently $\omega_i = 1/k$.
$\mathcal{D}_i \subset S_i$	Finite evaluation state set of task i , $ \mathcal{D}_i = D$. Both values are compared only on these states.
$[x]_+ = \max\{0, x\}$	Hinge, used to make every penalty one-sided.

Three collisions in the earlier draft are removed by this table. First, P_i against p_i^π . Second, the parameter vector of π_g is

ϕ and never θ . Third, J_i^* carries a task index and no argument, while $J_k(\cdot)$ carries a horizon index and one policy argument, so J_k^* and $J_k(\pi_g)$ are different objects and cannot be confused.

The objective with stored experts

The expert of task i is the best policy we ever produced for that task, so its score bounds what the global policy can reasonably be asked to reach,

$$J_k(\pi_g) \leq J_{1:k}^* \quad \text{whenever} \quad V_i^{\pi_g} \leq J_i^* \quad \text{for every } i \leq k. \quad (34)$$

Consolidation is therefore a problem of closing the gap to a known ceiling rather than a game against a moving opponent. Define the per-state gap on task i ,

$$g_i(s; \phi) = V_i^*(s) - V_i^{\pi_\phi}(s), \quad s \in \mathcal{D}_i, \quad (35)$$

and the one-sided squared shortfall of task i ,

$$F_i(\phi) = \frac{1}{D} \sum_{s \in \mathcal{D}_i} \left([g_i(s; \phi)]_+ \right)^2. \quad (36)$$

The hinge keeps the penalty one-sided, exactly as (9) did. If the global policy is already at or above the expert on a state, that state contributes nothing. The consolidation problem is the weighted sum over every task seen so far, the current task included,

$$\min_{\phi} \mathcal{L}_k(\phi) = \sum_{i=1}^k \omega_i F_i(\phi). \quad (37)$$

(37) is a minimization of shortfall and not a maximization of return, which is why no constraint is needed to stop the global policy from sacrificing one task for another. A well trained expert should already have a high score in one particular task. The global model should not be able to outperform it by a large margin and trade its performance to hide poor performance in some other tasks.

Global update

Fix $s \in \mathcal{D}_i$. Conditioning (3) on $s_0 = s$ removes the ρ_i factor,

$$p_i^{\pi_\phi}(\tau | s) = \prod_{t=0}^T \pi_\phi(a_t | s_t) P_i(s_{t+1} | s_t, a_t), \quad s_0 = s, \quad (38)$$

so that

$$\begin{aligned} \log p_i^{\pi_\phi}(\tau | s) &= \sum_{t=0}^T \log \pi_\phi(a_t | s_t) + \sum_{t=0}^T \log P_i(s_{t+1} | s_t, a_t), \\ \nabla_\phi \log p_i^{\pi_\phi}(\tau | s) &= \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t) + \underbrace{\sum_{t=0}^T \nabla_\phi \log P_i(s_{t+1} | s_t, a_t)}_0 \\ &= \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t). \end{aligned} \quad (39)$$

The product signs in the earlier version of this step were typographical. The logarithm turns the product of (38) into a sum, so (39) is a sum of score terms, matching (14) exactly.

Repeating (13)–(15) with ρ_i absent gives the state-conditioned policy gradient

$$\nabla_\phi V_i^{\pi_\phi}(s) = \mathbb{E}_{\tau \sim p_i^{\pi_\phi}(\cdot | s)} \left[G_i(\tau) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t) \right]. \quad (40)$$

The expert critic is a frozen network, hence

$$\nabla_\phi V_i^*(s) = 0 \quad \text{and} \quad \nabla_\phi g_i(s; \phi) = -\nabla_\phi V_i^{\pi_\phi}(s). \quad (41)$$

The squared hinge is continuously differentiable, with $\frac{d}{dx}([x]_+)^2 = 2[x]_+$, so

$$\begin{aligned}\nabla_\phi F_i(\phi) &= \frac{1}{D} \sum_{s \in \mathcal{D}_i} 2 [g_i(s; \phi)]_+ \nabla_\phi g_i(s; \phi) \\ &= -\frac{2}{D} \sum_{s \in \mathcal{D}_i} [g_i(s; \phi)]_+ \nabla_\phi V_i^{\pi_\phi}(s),\end{aligned}\tag{42}$$

and therefore

$$\nabla_\phi \mathcal{L}_k(\phi) = -\frac{2}{D} \sum_{i=1}^k \omega_i \sum_{s \in \mathcal{D}_i} [V_i^*(s) - V_i^{\pi_\phi}(s)]_+ \nabla_\phi V_i^{\pi_\phi}(s).\tag{43}$$

Descending $\mathcal{L}_k$ is thus ascending a *gap-weighted* policy gradient. Every state carries its own scalar weight $[g_i(s; \phi)]_+$, which is large where the global policy is far below its expert, and exactly zero where it has caught up.

Estimators. Draw N rollouts $\tau^{(1)}, \dots, \tau^{(N)}$ from each $s \in \mathcal{D}_i$ under π_ϕ in environment i . The same batch supplies both the weight and the direction,

$$\widehat{V}_i^{\pi_\phi}(s) = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}),\tag{44}$$

$$\overline{\nabla_\phi V_i^{\pi_\phi}}(s) = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t^{(n)} | s_t^{(n)}), \quad s_0^{(n)} = s.\tag{45}$$

Writing $\widehat{g}_i(s) = V_i^*(s) - \widehat{V}_i^{\pi_\phi}(s)$, the global update is

$$\phi^{(m+1)} = \phi^{(m)} + \frac{2\beta}{D} \sum_{i=1}^k \omega_i \sum_{s \in \mathcal{D}_i} [\widehat{g}_i(s)]_+ \overline{\nabla_\phi V_i^{\pi_\phi}}(s).\tag{46}$$

There is no multiplier update, because (37) is unconstrained.

Variance. Replacing $G_i(\tau^{(n)})$ in (45) by an advantage estimate $\widehat{A}_i(s_t, a_t)$ leaves (40) unbiased and is the version to implement. The weight $[\widehat{g}_i(s)]_+$ must still be built from the return estimate (44), or from a separately trained global critic, because it is a value and not an advantage.

Cost of the estimator. The expert side of (43) costs one forward pass of a frozen network per state, so all expert rollouts are gone. The global side still needs samples, because $\nabla_\phi V_i^{\pi_\phi}(s)$ in (40) is an expectation under π_ϕ that no stored network can supply. The saving is therefore real but one-sided, and the claim that rollouts become rare holds only for the expert values. If the frequency of global rollouts also has to drop, a global critic $V_\phi^c(s)$ trained by regression alongside the actor can supply $\widehat{V}_i^{\pi_\phi}(s)$ for the weight, while the direction still comes from a smaller batch of samples.

Algorithm, one consolidation phase after task k .

1. Freeze the finished local model as π_k^* and store its critic as V_k^* .
2. For each $i \leq k$, build $\mathcal{D}_i$ and cache $V_i^*(s)$ for every $s \in \mathcal{D}_i$ in one forward pass.
3. Repeat until the weighted shortfall $\mathcal{L}_k$ plateaus.
 - (a) Sample a task i with probability ω_i and a minibatch $\mathcal{B} \subset \mathcal{D}_i$.
 - (b) For each $s \in \mathcal{B}$, roll out π_ϕ in environment i from s and form (44) and (45).
 - (c) Apply (46) restricted to $\mathcal{B}$.
4. Log per-task $\widehat{V}_i^{\pi_\phi}$ against J_i^* for every $i \leq k$. This is the forgetting curve.

Scratchpad, questions and answers

Q1. Does the head-chopping trick fix the shared trunk without retraining the experts? Yes for the expert side, and it is exact rather than approximate. Deep-copy the saved shared-trunk checkpoint of task i twice. In copy A keep the policy head and drop the value head, in copy B keep the value head and drop the policy head. The two copies own separate storage after the copy, so a gradient into A can never reach B. Freeze B and it is V_i^* forever. Nothing is lost, because in a shared-trunk agent the value head of the checkpoint *is* the expert critic. The identical trunk weights are not a problem. They are the point, since the expert’s value function was always computed from those features.

Two boundaries. First, the trick only recovers experts if the end-of-task checkpoints still exist. It reconstructs, it does not resurrect. Second, and more important, it does not help the *global* side. A frozen copy of the global trunk is stale the moment ϕ moves, so $V_i^{\pi_g}(s)$ cannot come from a frozen head. Either estimate it from rollouts as in (44), or keep a global critic that is genuinely trained during consolidation on a trunk the actor gradient does not touch. The bug in the code was the third option, reading V^{π_g} off a head that was being dragged along by the actor update.

Q2. Why does the min-max formulation collapse? The saddle point of (12) existed because the two policies were each other’s only reference, one strong on the current task and weak on the past, the other the reverse. Storing every expert supplies an external, fixed reference for every task at once. The maximization over π_g and the minimization over μ then have nothing to trade, and (37) is a plain weighted regression onto the expert ceiling. Keeping the multiplier while also using stored experts, which is what the current code does, imposes a constraint whose reference is already an upper bound, so the constraint is either always slack or always violated depending on the sign of the gap.

Q3. Are 50 no-op plus 50 random intermediate states enough? No principled answer without measurement, but there is a check. The weight in (46) is $[\hat{g}_i(s)]_+$, so the estimator is only useful while the standard error of $\hat{V}_i^{\pi_\phi}(s)$ is small relative to the gap it is estimating. Log $\text{SE}(\hat{V}_i^{\pi_\phi}(s))/[\hat{g}_i(s)]_+$ and sweep D until that ratio stops shrinking. Separately, $\mathcal{D}_i$ built from no-op resets plus uniform intermediate states covers the *expert’s* visitation, not the global policy’s. A global policy that has drifted off the expert’s manifold will look fine on $\mathcal{D}_i$ and still fail on-policy, so $\mathcal{D}_i$ should be refreshed with states visited by π_ϕ as well, and the per-task evaluation in step 4 of the algorithm should stay a full rollout evaluation rather than a $\mathcal{D}_i$ average.

Q4. Is the comparison $V_i^*(s)$ against $\hat{V}_i^{\pi_\phi}(s)$ on the same scale? This has to be checked before anything else, and it is the most likely source of a silent bug. The stored critic was trained under whatever reward clipping, return normalization, bootstrapping horizon, and γ the expert run used, whereas (44) is an unnormalized truncated Monte Carlo return. If the expert used clipped rewards or a normalized value target, V_i^* and $\hat{V}_i^{\pi_\phi}$ live on different scales and the gap is meaningless. Fix by recording the normalization statistics with each expert checkpoint and inverting them, and by sanity-checking that $V_i^*(s) \approx \hat{V}_i^{\pi_i^*}(s)$ on a handful of states rolled out under the expert actor. That sanity check is the one place where the expert *actor* is still needed, which is a reason to store it even though the objective does not use it.

Q5. Squared hinge or plain hinge? The square in (36) makes a task with a large gap dominate the sum quadratically, which is useful early and harmful late, when one hard task can hold the update hostage. A plain hinge $\sum_i \omega_i \frac{1}{D} \sum_s [g_i(s)]_+$ gives every lagging state a weight of one and turns (46) into an unweighted policy gradient over the lagging set. Worth running both. The derivation is unchanged apart from dropping the factor $2[g_i(s)]_+$ down to $1\{g_i(s) > 0\}$.

Q6. Does the value gap certify behavioural absorption? Still open, and unchanged from the note in the previous section. Matching $V_i^*(s)$ at every $s \in \mathcal{D}_i$ constrains a scalar per state and not the action distribution, so two policies with equal values can act differently. Check empirically by measuring the action agreement between π_g and π_i^* along expert rollouts. If values match while agreement is low, add a distillation term $\text{KL}(\pi_i^*(\cdot | s) \| \pi_\phi(\cdot | s))$ on $\mathcal{D}_i$, which also removes the need for global rollouts entirely and turns consolidation into supervised policy distillation with a value-gap regularizer. This is the fallback if (46) is too noisy in practice.

Correction log against the handwritten pass.

1. $\gamma \in \{0, 1\}$ was written as a set. It is the open interval, $\gamma \in (0, 1)$, as in (1).
2. $\pi_g = \pi_\phi$ was annotated with θ as its parameter vector. The parameter vector of the global policy is ϕ ; θ belongs to the local policy.
3. $M_i = (S_i, A_i, \dots)$ was introduced with per-task states S_i but a shared action set A . Both are indexed here. If the Atari suite is action-set aligned, drop both indices and revert to (S, A) as in (1), but do not index one and not the other.
4. In the log-gradient of the trajectory law, three product signs should be sums. $\log \prod_t \pi = \sum_t \log \pi$, the transition term needs its own $\sum_t$, and the resulting score is $\sum_t \nabla_\phi \log \pi_\phi$. See (39).

5. $V_{\pi_i}^*$, $V_k^{\pi_g}(s)$ and $V_i(s)$ were used for the same family of objects with the policy sometimes in the subscript. Subscript is the task and superscript is the policy throughout, with $\star$ reserved for the frozen expert critic.
6. The trajectory law was written with $\rho_i(s_0)$ while the states s came from a fixed evaluation set. Those are different measures. Use the conditional law (38) when starting from a chosen s .
7. A stray comma terminated the align environment of (24) in the previous section. Remove it.